ANÁLISE E OTIMIZAÇÃO DE CONSULTAS SQL COM BASE EM LOGS DE EXECUÇÃO EM BANCOS DE DADOS RELACIONAIS

Arthur Meira[1], Marcel Campos Inocencio[2]

Resumo: Consultas SQL lentas podem gerar gargalos e comprometer o desempenho de sistemas que utilizam bancos de dados relacionais. Este trabalho teve como objetivo desenvolver e avaliar um protótipo para auxiliar na identificação e na análise preventiva de consultas com possíveis problemas de desempenho. Para avaliar a solução, construiu-se um banco experimental com dados fictícios de clientes, produtos, pedidos, itens e pagamentos, totalizando 4.648.714 registros. Executou-se uma carga composta por 12 consultas e 730 chamadas, cujas estatísticas foram coletadas no PostgreSQL e relacionadas aos planos de execução. O protótipo identificou sinais como leituras extensas de tabelas, funções aplicadas em filtros e diferenças entre a quantidade prevista e a quantidade histórica de linhas. Os resultados mostraram que tempo total, tempo médio e frequência destacam consultas diferentes. Na consulta de pedidos por ano, a alteração do filtro aproximou a quantidade prevista de linhas da referência histórica, sem comprovar redução de tempo. Na busca por e-mail, a comparação direta permitiu utilizar um índice existente e reduziu o tempo médio no servidor de 26,33 para 0,0303 milissegundo em uma rodada de 100 chamadas de cada versão. Os primeiros resultados comparados foram iguais, e a validade da alteração ficou condicionada à normalização dos e-mails da base. Também se desenvolveu uma interface web para apresentar o histórico, os planos e as sugestões. Concluiu-se que a combinação dessas informações pode apoiar a investigação de consultas lentas, desde que as alterações sejam verificadas por testes de resultado e desempenho.

Palavras-chave: bancos de dados relacionais; consultas SQL; otimização de consultas; estatísticas de execução; planos de execução.

ABSTRACT: Slow SQL queries can create bottlenecks and impair the performance of systems that use relational databases. This study aimed to de-

[1]Curso de Ciência da Computação, Universidade do Extremo Sul Catarinense (Unesc), Criciúma – Santa Catarina – Brasil. arthurmeiraprog@unesc.net

[2]Orientador, Curso de Ciência da Computação, Universidade do Extremo Sul Catarinense (Unesc), Criciúma – Santa Catarina – Brasil. marcel.inocencio@gmail.com

velop and evaluate a prototype to support the identification and preventive analysis of queries with potential performance problems. An experimental database containing fictional customers, products, orders, order items, and payments was built, totaling 4,648,714 records. A workload comprising 12 queries and 730 calls was executed, and PostgreSQL statistics were collected and associated with execution plans. The prototype identified warning signs such as extensive table scans, functions applied to filters, and differences between estimated and historical row counts. The results showed that total time, mean time, and execution frequency highlight different queries. For the query selecting orders by year, changing the filter brought the estimated row count closer to the historical reference without demonstrating a runtime reduction. For the email lookup, a direct comparison enabled an existing index and reduced mean server execution time from 26.33 to 0.0303 milliseconds in a run of 100 calls per version. The first result sets compared were equal, and the validity of the change depended on the normalization of email values in the database. A web interface was also developed to present historical statistics, execution plans, and suggestions. The study concluded that combining this information can support the investigation of slow queries, provided that proposed changes are verified through result and performance tests.

Keywords: relational databases; SQL queries; query optimization; execution statistics; execution plans.

1 INTRODUÇÃO

A linguagem SQL (*Structured Query Language*) permite consultar e manipular informações armazenadas em bancos de dados relacionais. Ao escrever uma consulta, o desenvolvedor informa quais dados deseja obter, enquanto o sistema gerenciador de banco de dados (SGBD) determina como a busca será realizada. Uma mesma consulta pode ser processada por diferentes caminhos, dependendo dos filtros, dos índices, das relações entre as tabelas e da quantidade de registros envolvidos (LAN; BAO; PENG, 2021).

A escolha do caminho de execução é realizada pelo otimizador. Esse componente compara alternativas e seleciona um plano com as operações necessárias para produzir o resultado. O plano pode incluir leituras de tabelas, acessos por índice, junções, agrupamentos e ordenações. Embora duas estratégias retornem os mesmos dados, elas podem exigir

quantidades diferentes de processamento e apresentar tempos distintos.

De acordo com Lan, Bao e Peng (2021), a otimização envolve a previsão da quantidade de registros, a estimativa do custo das operações e a busca por alternativas de plano. A previsão da quantidade de registros é chamada de estimativa de cardinalidade. Quando ela se distancia do volume efetivamente retornado, o banco pode comparar as alternativas a partir de informações pouco representativas.

As propriedades dos dados também influenciam esse processo. Kossmann, Papenbrock e Naumann (2022) discutem como restrições de unicidade e relações entre atributos podem apoiar a otimização. Uma coluna que não permite valores repetidos, por exemplo, fornece uma informação útil sobre o resultado de uma busca. Essas propriedades precisam ser válidas no conjunto analisado, pois uma alteração baseada em uma suposição incorreta pode modificar a resposta esperada.

Além do plano, é necessário observar o comportamento das consultas ao longo das execuções. Uma consulta demorada executada uma única vez apresenta um impacto diferente de uma consulta rápida repetida milhares de vezes. Informações sobre frequência, tempo acumulado, tempo médio e quantidade de linhas retornadas ajudam a selecionar os casos que merecem investigação.

A pesquisa se justifica pela necessidade de identificar e reduzir gargalos causados por consultas SQL lentas. Esses gargalos podem aumentar o tempo de resposta das aplicações, elevar o consumo de recursos e prejudicar vários usuários. Sem um processo de análise, mudanças podem ser aplicadas sem evidências suficientes, enquanto consultas com maior impacto permanecem sem investigação.

Nesse contexto, formulou-se a seguinte questão de pesquisa: como combinar estatísticas históricas e planos de execução para identificar consultas SQL potencialmente problemáticas, explicar os sinais encontrados e apoiar a validação de melhorias em um banco de dados relacional?

O objetivo geral foi desenvolver e avaliar um processo automatizado de análise preventiva de consultas SQL baseado em estatísticas históricas e planos de execução. Os objetivos específicos foram construir uma base experimental, executar consultas com diferentes características, coletar e classificar suas métricas, identificar sinais de atenção nos planos, comparar consultas originais e alternativas, validar uma alteração e apresentar as informações em uma interface visual.

A contribuição do trabalho está na integração dessas atividades

em uma ferramenta de apoio. O protótipo organiza as evidências e apresenta sugestões para investigação. As alterações não são aplicadas automaticamente, e a decisão de adotá-las depende da verificação dos resultados e do desempenho nas condições de uso.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 OTIMIZAÇÃO E PLANOS DE EXECUÇÃO

O otimizador transforma uma instrução SQL em um plano executável. Conforme Lan, Bao e Peng (2021), a escolha depende da relação entre as estimativas de linhas, o modelo de custos e as alternativas consideradas. Uma previsão inadequada pode influenciar a ordem das junções ou a escolha entre ler uma tabela inteira e utilizar um índice.

Em uma retrospectiva sobre a avaliação dos otimizadores, Leis et al. (2025) destacam a importância das estimativas e das características das cargas utilizadas nos experimentos. A discussão reforça a necessidade de observar o comportamento das consultas em situações concretas, pois uma decisão adequada para determinada distribuição de dados pode não ser a melhor em outro cenário.

No PostgreSQL, o plano possui uma estrutura de árvore. Cada nó representa uma operação, como `Seq Scan`, para leitura sequencial, `Index Scan`, para acesso por índice, e `Hash Join`, para um método de junção. O plano apresenta também custos e quantidades estimadas de linhas (PostgreSQL Global Development Group, s.d.d).

O custo é uma unidade interna usada pelo planejador para comparar alternativas. Ele não corresponde a uma duração em milissegundos. O comando `EXPLAIN`, sem `ANALYZE`, apresenta o plano estimado sem executar a consulta candidata. Já `EXPLAIN ANALYZE` executa a instrução e acrescenta informações observadas. Essa diferença orientou a separação entre análise preventiva e medição neste trabalho.

2.2 ESTATÍSTICAS E AVALIAÇÃO DE DESEMPENHO

Huang et al. (2023) organizam a pesquisa sobre desempenho de bancos de dados em atividades como previsão, diagnóstico e ajustes. Essa distinção ajuda a compreender que reconhecer um possível problema é uma etapa diferente de aplicar uma alteração e verificar seu efeito.

A extensão `pg_stat_statements` disponibiliza estatísticas acumuladas de comandos executados no PostgreSQL. Entre seus campos estão a quantidade de chamadas, o tempo total, o tempo médio e o número

de linhas retornadas ou afetadas. Comandos de estrutura equivalente podem compartilhar uma entrada, mesmo quando os valores utilizados nos filtros são diferentes (PostgreSQL Global Development Group, s.d.b).

Tempo total, tempo médio e frequência respondem a perguntas distintas. O tempo total permite observar o consumo acumulado; o tempo médio descreve a duração média de uma chamada; e a frequência mostra quais comandos aparecem mais vezes. A quantidade de linhas ajuda a interpretar essas medidas, pois retornar milhões de registros exige trabalho diferente de localizar um único cliente.

Neste estudo, essas informações foram tratadas como estatísticas históricas. Não se realizou a análise de arquivos de log contendo uma linha separada para cada execução. Essa delimitação é importante porque os dados agregados não permitem reconstruir todos os detalhes de cada chamada.

2.3 ÍNDICES E REESCRITA DE CONSULTAS

Índices oferecem caminhos de acesso que podem reduzir a quantidade de dados examinados em determinadas consultas. Seu benefício depende dos filtros, do volume selecionado e das características da base. Abbasi et al. (2024) investigam estratégias de indexação no processamento de eventos complexos, mostrando a relevância de avaliar índices em conjunto com a carga de consultas. Embora esse cenário seja diferente do presente experimento, ele reforça a necessidade de verificar o efeito das escolhas de indexação.

Uma leitura sequencial não representa, isoladamente, uma falha. Ela pode ser adequada quando uma consulta precisa retornar grande parte da tabela. Da mesma forma, um índice não garante melhoria em todas as operações e exige manutenção quando os dados são inseridos ou alterados.

A aplicação de funções sobre colunas filtradas também merece atenção. Um índice sobre `email` não equivale a um índice sobre a expressão `lower(email)`. A documentação do PostgreSQL apresenta índices de expressão como uma possibilidade para esse tipo de busca (PostgreSQL Global Development Group, s.d.a).

Outra alternativa é reescrever o filtro, desde que o resultado seja preservado. Uma busca por e-mail pode utilizar comparação direta quando os dados já estão normalizados. Um filtro por ano pode ser escrito como um intervalo de datas. A validade dessas mudanças depende das regras

do banco e não pode ser determinada apenas pela redução de um custo estimado.

3 MATERIAIS E MÉTODOS

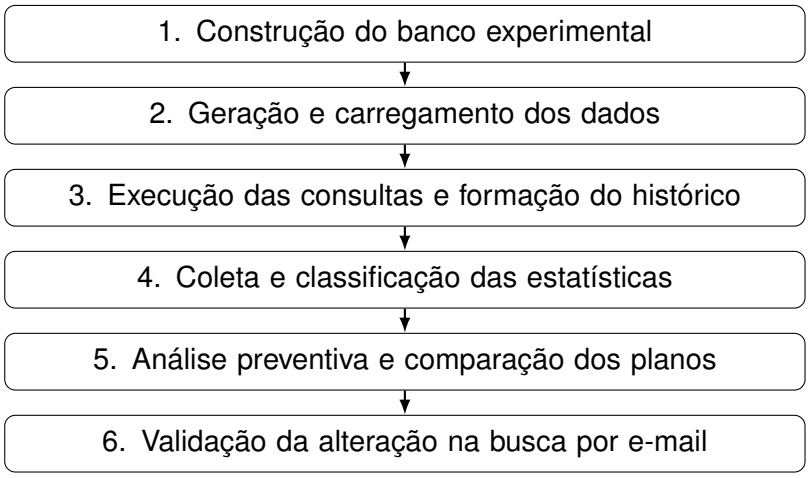
3.1 ETAPAS DO DESENVOLVIMENTO

Iniciou-se a parte prática com a construção de um protótipo para executar consultas, coletar estatísticas, analisar planos e apresentar os resultados. O estudo foi realizado em ambiente controlado, utilizando dados fictícios e uma carga definida para o experimento.

O desenvolvimento foi organizado em seis etapas. Primeiro, construiu-se o banco experimental. Depois, geraram-se os dados e realizou-se seu carregamento. Na terceira etapa, executaram-se as consultas para produzir o histórico. Na quarta, coletaram-se e classificaram-se as estatísticas. Na quinta, analisaram-se os planos estimados. Por fim, compararam-se alternativas e realizou-se a validação da busca por e-mail.

A Figura 1 apresenta essa sequência. A comparação de planos foi utilizada nos casos de pedidos por ano e busca por e-mail. A execução para medir o efeito da alteração foi realizada somente no segundo caso.

Figura 1 – Etapas da solução desenvolvida



Fonte: elaboração própria.

3.2 AMBIENTE DE DESENVOLVIMENTO

A primeira rodada experimental utilizou PostgreSQL 18.1, Python 3.14.3 e Psycopg 3.3.6 em Windows de 64 bits. O PostgreSQL armazenou os dados, executou as consultas e forneceu os planos. O Python foi utilizado na geração dos registros, na execução da carga e na organização dos resultados. O Psycopg realizou a comunicação entre a aplicação e o

banco.

O DBeaver auxiliou na administração e na conferência das tabelas. Também foi criado um ambiente virtual Python para separar as dependências do projeto das demais instalações do computador. As credenciais foram mantidas na configuração local, sem inclusão nos trechos de código apresentados no artigo.

Em uma rodada posterior de verificação, utilizou-se PostgreSQL 18.6 e Python 3.14.7. Nessa ocasião, verificou-se um ambiente com processador AMD Ryzen 5 5600X, aproximadamente 32 GB de memória RAM, SSD Kingston NV2 de 2 TB e Windows 11 Pro. Essa configuração descreve o ambiente disponível na revisão; não houve registro equivalente que permitisse caracterizar integralmente as condições físicas da primeira rodada.

Os resultados da primeira rodada foram mantidos separados das verificações posteriores. Essa distinção foi necessária porque versões do software, uso da memória e atividades de outros programas podem influenciar os tempos.

Os arquivos foram organizados em pastas para os scripts SQL, os programas Python, a interface, os testes, a documentação e os resultados. Essa organização permitiu executar separadamente a geração dos dados, a carga de consultas, a coleta e a análise preventiva.

3.3 BANCO EXPERIMENTAL E VOLUMES UTILIZADOS

Criou-se o banco `tcc_sql_optimizer` com cinco tabelas: `clientes`, `produtos`, `pedidos`, `itens_pedido` e `pagamentos`. Um cliente pode realizar vários pedidos. Cada item relaciona um pedido a um produto, enquanto o pagamento referencia o pedido correspondente.

Todas as tabelas receberam chaves primárias e os relacionamentos foram definidos por chaves estrangeiras. A coluna de e-mail dos clientes recebeu uma restrição de unicidade, associada a um índice. Não foram acrescentados índices específicos para favorecer as consultas do experimento.

O gerador produziu dois ou três itens e um pagamento para cada pedido. Essas quantidades descrevem a base gerada. Em particular, a existência de um único pagamento por pedido foi uma regra do programa de geração, e não uma restrição de unicidade imposta à chave estrangeira de pagamentos.

A Tabela 1 apresenta o volume obtido e a finalidade das tabelas.

Tabela 1 – Composição da base experimental

Tabela	Registros	Finalidade
Clientes	100.000	Buscas por identificador e e-mail; associação aos pedidos.
Produtos	50.000	Filtros por categoria, preço e situação.
Pedidos	1.000.000	Filtros temporais, junções e agrupamentos.
Itens de pedido	2.498.714	Associação entre pedidos e produtos; respostas volumosas.
Pagamentos	1.000.000	Associação aos pedidos e consulta das situações de pagamento.
Total	**4.648.714**	Base utilizada nas consultas do experimento.

Fonte: elaboração própria.

As quantidades foram definidas para combinar buscas seletivas com operações que percorrem grandes volumes. Cem mil clientes permitem contrastar a busca por identificador ou e-mail com uma leitura ampla da tabela. Um milhão de pedidos torna os filtros de data e as junções relevantes para a análise. A quantidade de itens, superior à de pedidos, permite observar o efeito de respostas extensas.

Esses volumes foram escolhas de dimensionamento do experimento. Não representam uma amostra estatística de uma empresa nem um padrão universal para avaliação de bancos de dados.

3.4 GERAÇÃO E CARREGAMENTO DOS DADOS

O gerador foi implementado em Python, no arquivo `scripts/populate_database.py`. O programa recebeu as quantidades de clientes, produtos e pedidos, o tamanho dos lotes e uma semente para os sorteios. Os valores padrão foram 100.000 clientes, 50.000 produtos, 1.000.000 de pedidos, lotes de 10.000 e semente 2026.

A semente inicia o gerador de números pseudoaleatórios. Mantidos o programa, o ambiente e os parâmetros, ela permite repetir a sequência de escolhas utilizada na geração. Essa característica auxilia a reconstrução dos dados, mas não garante tempos de execução idênticos.

O Código 1 apresenta um recorte das decisões utilizadas na geração dos pedidos. O trecho depende das funções e das variáveis do programa original.

Código 1: Recorte da geração dos pedidos

```
1 rng = random.Random(args.seed)
```

```
2
3  customer_id = popular_id(
4      rng, customer_count, 0.20, 0.80
5  )
6
7  year = rng.choices(
8      (2024, 2025, 2026),
9      weights=(20, 30, 50),
10     k=1
11 )[0]
12
13 status = rng.choices(
14     ("pendente", "pago", "enviado", "entregue", "cancelado"),
15     weights=(12, 13, 20, 48, 7),
16     k=1
17 )[0]
```

Fonte: elaboração própria, adaptado de `scripts/populate_database.py`.

A função `popular_id` concentrou 80% da probabilidade de escolha de clientes nos primeiros 20% dos identificadores. Para os produtos, 70% da probabilidade foi concentrada nos primeiros 10% do catálogo. Essas escolhas criaram grupos com atividades diferentes, permitindo examinar consultas que selecionam parcelas distintas da base.

Os anos de 2024, 2025 e 2026 receberam pesos de 20, 30 e 50. Os pesos representam probabilidades e não quantidades exatas de registros. Em 2026, as datas foram geradas entre 1ž de janeiro e antes de 1ž de setembro.

Cada pedido recebeu dois ou três itens. O valor total foi calculado a partir das quantidades e dos valores unitários, e a situação do pagamento foi relacionada à situação do pedido. Os valores monetários foram calculados em centavos inteiros e convertidos para `Decimal`. Os e-mails foram gerados em letras minúsculas, condição utilizada posteriormente na avaliação da busca por e-mail.

O carregamento utilizou `COPY`, com envio de linhas pelo Psycopg. A documentação do PostgreSQL recomenda essa operação para cargas volumosas, e o Psycopg disponibiliza `write_row` para transmitir os registros (PostgreSQL Global Development Group, s.d.c; The Psycopg Team, s.d.).

O Código 2 mostra o carregamento dos pedidos e das tabelas relacionadas. O tamanho do lote controlou a quantidade de pedidos preparados em cada iteração; o lote correspondente de itens podia ser maior,

pois cada pedido possuía dois ou três itens.

Código 2: Carregamento dos pedidos em lotes

```
for first_id in range(1, args.pedidos + 1, args.lote):
    stop_id = min(
        first_id + args.lote,
        args.pedidos + 1
    )

    orders, items, payments = make_order_batch(
        first_id,
        stop_id,
        counts["itens_pedido"] + 1,
        args.clientes,
        prices,
        rng
    )

    copy_rows(cursor, "pedidos", orders)
    copy_rows(cursor, "itens_pedido", items)
    copy_rows(cursor, "pagamentos", payments)

    counts["itens_pedido"] += len(items)
```

Fonte: elaboração própria, adaptado de `scripts/populate_database.py`.

Antes da carga, o programa verificou o nome do banco e se as tabelas estavam vazias. Caso encontrasse registros existentes, interrompia o procedimento sem apagá-los. A confirmação da transação ocorreu ao final da carga completa.

Depois do carregamento, executou-se `ANALYZE` nas cinco tabelas para atualizar as estatísticas utilizadas pelo planejador, conforme a orientação da documentação (PostgreSQL Global Development Group, s.d.c). As consultas de conferência foram organizadas em `sql/validate_population.sql`, incluindo contagens e verificações de valores, datas e situações dos pedidos e pagamentos.

3.5 CONSULTAS E EXECUÇÃO DA CARGA

Foram elaboradas 12 consultas SQL com filtros, junções, agrupamentos, ordenações e subconsultas. Elas receberam os nomes Q01 a Q12 para organização local. Esses nomes não correspondem aos identificadores internos calculados pelo PostgreSQL.

A frequência variou entre uma e 500 chamadas. Essa diferença

foi definida para produzir consultas frequentes e seletivas, consultas intermediárias e operações pouco frequentes com respostas extensas. A Tabela 2 apresenta o conjunto.

Tabela 2 – Perfil da carga experimental

Consulta	Chamadas	Operação principal
Q01	500	Busca de cliente pela chave primária.
Q02	100	Filtros de produtos com ordenação e limite.
Q03	60	Consulta dos pedidos de um cliente.
Q04	20	Seleção dos pedidos cancelados.
Q05	15	Junção de pedidos e clientes com filtro pelo ano.
Q06	10	Junções entre as cinco tabelas e agrupamento.
Q07	8	Agregação mensal de pedidos.
Q08	6	Ordenação dos pedidos de maior valor.
Q09	5	Busca por e-mail com aplicação de `LOWER`.
Q10	3	Pedidos acima do valor médio calculado por subconsulta.
Q11	2	Contagem de pedidos por cliente com subconsulta.
Q12	1	Leitura de todos os itens de pedido.
Total	**730**	**Chamadas previstas para a carga.**

Fonte: elaboração própria.

Inicialmente, executou-se uma chamada de cada consulta para verificar seu funcionamento. Depois, realizou-se a carga completa com valores de filtros fixos. O programa organizou as repetições e embaralhou sua ordem com semente 2026. A sessão foi configurada como somente leitura e com limite de 120 segundos por consulta.

As linhas retornadas foram consumidas pelo programa, de modo que a medição do cliente incluiu o recebimento dos resultados. Não foram simulados usuários concorrentes.

O registro da primeira rodada não preservou o horário de reinicialização das estatísticas. Assim, a correspondência entre 730 chamadas previstas e observadas foi utilizada como conferência de contagem, sem ser tratada como prova completa do isolamento da janela.

3.6 COLETA DAS ESTATÍSTICAS E MEDIÇÃO DOS TEMPOS

A coleta utilizou a extensão `pg_stat_statements`, versão 1.12 na primeira rodada. Sua preparação incluiu o carregamento do módulo em `shared_preload_libraries`, a reinicialização do serviço e a criação da extensão no banco experimental. O cálculo dos identificadores de consulta permaneceu habilitado.

O coletor consultou as estatísticas do banco e do usuário utilizados no experimento. Em seguida, filtrou os comandos relacionados às tabelas experimentais. Foram preservados o identificador, o texto da consulta, a quantidade de chamadas, o tempo total, o tempo médio e as linhas retornadas.

Os campos `total_exec_time` e `mean_exec_time` representam tempos de execução em milissegundos. O campo `calls` registra a quantidade de chamadas, e `rows`, o número acumulado de linhas (PostgreSQL Global Development Group, s.d.b).

Os resultados foram organizados em classificações por tempo total, tempo médio e frequência. A multiplicação do tempo médio pelo número de chamadas reproduz aproximadamente o tempo total e, por isso, não foi tratada como uma medida independente.

Além dos dados do servidor, o programa mediu o intervalo observado pelo cliente com `perf_counter`, função do Python destinada à medição de intervalos (Python Software Foundation, s.d.). O Código 3 apresenta esse procedimento.

Código 3: Medição do tempo observado pelo cliente

```
from contextlib import closing
from time import perf_counter

started = perf_counter()
row_count = 0

with connection.cursor() as cursor:
    with closing(cursor.stream(query.statement)) as records:
        for _ in records:
            row_count += 1

elapsed_ms = (perf_counter() - started) * 1000
```

Fonte: elaboração própria, adaptado de `scripts/generate_workload.py`.

O primeiro registro de tempo ocorreu antes da execução. O segundo ocorreu depois de percorrer todas as linhas retornadas. A diferença corresponde ao intervalo observado pelo cliente, e a multiplicação por 1.000 converte segundos em milissegundos.

Esse tempo inclui comunicação, recebimento e processamento das linhas no Python. Portanto, foi mantido separado das métricas do PostgreSQL. A duração completa do programa também inclui preparação da sessão e apresentação do progresso, não sendo equivalente à soma dos

tempos de execução no servidor.

Implementou-se a exportação para CSV e JSON. Os resultados da primeira rodada foram documentados a partir das saídas de terminal informadas pelo autor, com valores arredondados. Os arquivos CSV e JSON originais dessa rodada não estavam disponíveis no conjunto documental conferido, o que limita a auditoria dos valores. Os arquivos das verificações posteriores pertencem a uma rodada posterior e foram tratados separadamente.

3.7 ANÁLISE PREVENTIVA DOS PLANOS

O analisador recebeu uma instrução `SELECT` e solicitou ao PostgreSQL um plano estimado em formato JSON, com `ANALYZE FALSE`. O programa percorreu a árvore do plano para identificar operações, relações, filtros e quantidades estimadas.

A associação ao histórico utilizou o identificador da consulta fornecido pelo plano, com seleção do banco e do usuário da conexão. Quando não foi encontrada uma entrada correspondente, a análise manteve apenas as estimativas, sem atribuir tempo ou quantidade histórica inexistentes. A média de linhas foi obtida pela divisão do total histórico de linhas pelo número de chamadas. Essa associação descreve o contexto consultado e não pressupõe que o identificador permaneça igual entre ambientes distintos.

Foram rejeitados comandos de alteração, múltiplas instruções, `SELECT INTO` e instruções iniciadas por `WITH`, conforme o escopo do validador implementado. Essas verificações restringem a entrada, mas não constituem um analisador sintático completo para qualquer expressão SQL.

As regras procuraram sinais como seleção de todas as colunas, leitura sequencial com filtro em tabela grande, função aplicada ao filtro, ordenação extensa, subplanos e retorno volumoso. Também foi observada a diferença entre as linhas previstas no plano e a média histórica de linhas por chamada.

A Tabela 3 resume os critérios implementados. Os limites e os pontos foram escolhas do protótipo para organizar os casos; não foram obtidos por treinamento estatístico.

Tabela 3 – Critérios utilizados pelo analisador

Sinal	Condição observada	Pontos
Seleção ampla	Presença de `SELECT *`.	1
Varredura sequencial filtrada	Relação com pelo menos 50.000 linhas estimadas.	3
Função no filtro	Expressões como `LOWER` e `EXTRACT`.	2
Ordenação extensa	Pelo menos 10.000 linhas estimadas.	2
Laço aninhado	Produto das estimativas dos filhos de pelo menos 1.000.000.	2
Resultado volumoso	Pelo menos 100.000 linhas de saída estimadas.	3
Divergência de linhas	Diferença de pelo menos dez vezes entre estimativa e histórico, para valores positivos.	3
Subplano	Presença de uma operação identificada como subplano.	2

Fonte: elaboração própria.

A soma dos pontos foi limitada a dez. Valores de zero a dois foram classificados como atenção baixa; de três a seis, média; e de sete a dez, alta. Essa classificação sintetiza os sinais encontrados e não representa uma previsão de duração.

Cada sinal foi acompanhado de evidência e sugestão. Uma leitura sequencial gerou orientação para examinar seletividade e índices. Uma função aplicada ao filtro gerou uma indicação para estudar uma reescrita ou um índice de expressão. Nenhuma recomendação alterou automaticamente o banco.

3.8 COMPARAÇÃO E VALIDAÇÃO DAS ALTERNATIVAS

Selecionaram-se dois casos: a consulta de pedidos por ano e a busca por e-mail. Na primeira, comparou-se a extração do ano com um intervalo de datas. A avaliação foi apenas preventiva, sem executar a alternativa para medir seu tempo.

Na busca por e-mail, comparou-se `LOWER(email)` com uma comparação direta. A alternativa poderia utilizar o índice existente porque os e-mails foram gerados em letras minúsculas. Essa condição foi considerada parte da justificativa da reescrita.

O programa `scripts/validate_recommendation.py` executou 100 chamadas de cada versão, intercaladas em ordem pseudoaleatória com semente 2026. O primeiro conjunto de resultados de cada versão foi armazenado e comparado. Nas demais chamadas, foram contabilizadas

as linhas e os tempos.

Como as estatísticas são acumuladas, os contadores foram lidos antes e depois da rodada. O Código 4 mostra como o programa obteve o tempo médio do intervalo, com nomes de variáveis adaptados para facilitar a leitura.

Código 4: Cálculo do tempo médio da rodada

```
chamadas = depois["calls"] - antes["calls"]

tempo_total_ms = (
    depois["total_exec_time"]
    - antes["total_exec_time"]
)

tempo_medio_ms = (
    tempo_total_ms / chamadas
    if chamadas > 0
    else 0.0
)
```

Fonte: elaboração própria, adaptado de `scripts/validate_recommendation.py`.

Primeiro, subtraiu-se o número inicial de chamadas do número final. O mesmo procedimento foi aplicado ao tempo acumulado. Depois, dividiu-se o tempo do intervalo pela quantidade de chamadas. Por exemplo, 2.633 milissegundos acumulados em 100 chamadas correspondem a uma média de 26,33 milissegundos.

Esse procedimento pressupõe que os contadores não sejam reinicializados durante a rodada e que outras execuções das mesmas consultas não sejam incluídas no intervalo. O teste foi realizado no banco experimental local, sem simulação de usuários concorrentes.

A comparação dos primeiros resultados verificou o caso utilizado no experimento. Ela não constitui uma prova de equivalência para todas as entradas possíveis. A intercalação das versões distribuiu as chamadas ao longo da rodada, mas não eliminou a influência dos dados já disponíveis em memória.

3.9 INTERFACE E VERIFICAÇÃO FUNCIONAL

Desenvolveu-se uma interface web local, disponibilizada em `127.0.0.1:8765`. A aplicação utilizou os mesmos módulos de coleta e análise empregados pela linha de comando.

A visão geral apresenta os totais do experimento e o histórico

das consultas. A área de análise permite escolher um exemplo ou informar um `SELECT`. A comparação exibe os planos da versão original e da alternativa. A seção didática explica os conceitos utilizados pelo protótipo.

Nas telas preventivas, informou-se que a consulta candidata não seria executada para medir tempo. O histórico real e as estimativas do plano foram apresentados em áreas distintas.

Foram realizados sete testes automatizados das respostas da interface, além de verificações de integração com o banco. Examinaram-se entradas inválidas, consultas sem histórico e a preservação dos contadores ao utilizar análise e comparação. Esses testes verificaram o funcionamento das operações, mas não constituíram uma avaliação de usabilidade com participantes.

4 RESULTADOS E DISCUSSÃO

4.1 EXECUÇÃO E CLASSIFICAÇÃO DAS CONSULTAS

O teste funcional executou as 12 consultas sem erro em aproximadamente 14,3 segundos no cliente Python. A carga completa realizou 730 chamadas em aproximadamente 103,2 segundos. O coletor selecionou as 12 consultas do experimento e excluiu outras entradas que não pertenciam ao conjunto analisado.

Os critérios destacaram consultas diferentes. A Q05 apresentou o maior tempo acumulado, com 42.472,08 milissegundos em 15 chamadas. A Q12 apresentou o maior tempo médio entre as consultas analisadas, com 6.795,63 milissegundos. Como houve apenas uma chamada, essa média corresponde a uma única observação e não caracteriza a variabilidade de execuções repetidas. A Q01 foi a mais frequente, com 500 chamadas e 9,25 milissegundos acumulados.

A Tabela 4 reúne as consultas destacadas pelos três critérios.

Tabela 4 – Consultas destacadas pelas estatísticas

Consulta	Chamadas	Total (ms)	Média (ms)	Destaque
Q05	15	42.472,08	2.831,47	Maior tempo acumulado.
Q12	1	6.795,63	6.795,63	Maior tempo médio; uma chamada.
Q01	500	9,25	0,02	Maior frequência.

Fonte: elaboração própria.

A Q01 mostrou que muitas chamadas não significam necessari-

amente alto consumo acumulado. A busca por chave primária permaneceu rápida no conjunto avaliado.

A Q12 retornou 2.498.714 linhas. Como foi executada apenas uma vez, seu tempo médio corresponde a essa observação e não a uma média obtida de várias repetições. O volume retornado também impede uma comparação direta com consultas que buscam um único registro.

A Q05 reuniu o maior tempo acumulado e um filtro que poderia ser examinado por meio do plano. Por esse motivo, foi selecionada para a análise da consulta de pedidos por ano.

4.2 CONSULTA DE PEDIDOS POR ANO

A Q05 selecionou pedidos de 2026 e os dados dos respectivos clientes. O filtro original aplicava `EXTRACT` à coluna de data, conforme o Código 5.

Código 5: Consulta de pedidos por ano

```
SELECT *
FROM public.pedidos p
JOIN public.clientes c ON c.id = p.cliente_id
WHERE EXTRACT(YEAR FROM p.data_pedido) = 2026;
```

Fonte: elaboração própria.

O plano original estimou 5.000 linhas de saída. O histórico registrou 7.495.140 linhas em 15 chamadas, equivalentes a uma média de 499.676 linhas por execução. Assim, a referência histórica ficou próxima de cem vezes a previsão do plano.

O analisador identificou seleção de todas as colunas, leitura sequencial com filtro, função aplicada à data e diferença elevada entre a estimativa e o histórico. O nível de atenção foi alto, com nove pontos.

Preparou-se uma alternativa que preservou a seleção e a junção, substituindo apenas o filtro por limites de data, apresentados no Código 6.

Código 6: Filtro alternativo por intervalo de datas

```
WHERE p.data_pedido >= TIMESTAMP '2026-01-01 00:00:00'
  AND p.data_pedido <  TIMESTAMP '2027-01-01 00:00:00'
```

Fonte: elaboração própria.

No plano da alternativa, a quantidade prevista passou para aproximadamente 499.434 linhas, valor próximo da média histórica. A Tabela 5 resume a comparação.

Tabela 5 – Comparação preventiva da consulta por ano

Informação	Original	Alternativa
Linhas estimadas	5.000	499.434
Nós no plano	5	4
Custo estimado	19.003,00	38.633,07
Nível de atenção	Alto, 9/10	Alto, 7/10

Fonte: elaboração própria.

O custo estimado aumentou, embora a previsão de linhas tenha se aproximado do histórico. Esse resultado não comprova que a alternativa ficou mais lenta, pois o custo não é tempo e os planos passaram a considerar quantidades de linhas diferentes.

Também não foi concluído que a consulta ficou mais rápida. A comparação solicitou apenas planos estimados. A alternativa permaneceu como uma possibilidade para validação posterior.

Esse caso ilustra a relação entre cardinalidade e escolha do plano discutida por Lan, Bao e Peng (2021). Ao comparar diretamente a coluna com limites de data, o planejador passou a apresentar uma previsão mais próxima do histórico disponível. O resultado é compatível com a importância atribuída às estimativas na literatura, mas não demonstra que a reescrita melhore qualquer filtro temporal. A distribuição dos dados e o intervalo selecionado permanecem determinantes.

A pontuação passou de nove para sete, mas permaneceu na faixa alta. Ela resume regras acionadas, não uma porcentagem de melhoria. A mudança do plano pode retirar alguns sinais e introduzir outros; portanto, a interpretação exige examinar as evidências de cada versão, sem usar a pontuação como substituta da medição.

A permanência de uma leitura ampla é compatível com o fato de a consulta selecionar aproximadamente metade dos pedidos. Portanto, a presença de uma varredura sequencial não foi tratada como motivo suficiente para recomendar um índice.

4.3 BUSCA POR E-MAIL

A Q09 aplicava `LOWER` sobre a coluna de e-mail. O plano original previa uma leitura sequencial da tabela de clientes. A alternativa utilizou comparação direta com o valor já normalizado, conforme o Código 7.

Código 7: Comparação dos filtros da busca por e-mail

```sql
-- Forma original
WHERE LOWER(email) = 'cliente123@example.com';

-- Alternativa avaliada
WHERE email = 'cliente123@example.com';
```

Fonte: elaboração própria.

O plano original apresentou `Seq Scan`, enquanto a alternativa apresentou `Index Scan`. O índice já existia em razão da restrição de unicidade da coluna. Não foi necessário criar uma nova estrutura para esse teste.

Os primeiros conjuntos de resultados comparados foram iguais. Cada versão retornou uma linha por chamada, totalizando 100 linhas em 100 execuções. A Tabela 6 apresenta os tempos médios da rodada.

Tabela 6 – Resultado da validação da busca por e-mail

Métrica	**Original**	**Alternativa**
Chamadas	100	100
Tempo total no servidor (ms)	2.633,00	3,03
Tempo médio no servidor (ms)	26,3300	0,0303
Tempo médio no cliente (ms)	26,7149	0,3964

Fonte: elaboração própria.

O tempo médio no servidor caiu de 26,33 para 0,0303 milissegundo. A redução calculada pelo programa foi de aproximadamente 99,88%. O tempo observado pelo cliente também diminuiu, mas permaneceu superior ao tempo registrado pelo servidor, pois inclui atividades de comunicação e processamento dos resultados.

Esse resultado confirmou o benefício da alteração no caso avaliado. A conclusão depende, entretanto, de os e-mails estarem em letras minúsculas. Caso a aplicação permita outras formas de armazenamento e exija comparação sem distinção entre maiúsculas e minúsculas, a retirada de `LOWER` pode mudar a resposta.

O experimento evidencia a diferença entre diagnóstico e validação. Na consulta por ano, verificou-se uma mudança no plano. Na busca por e-mail, além do plano, houve comparação do resultado e medição de execuções. Somente o segundo caso fundamentou uma conclusão de redução de tempo.

Na busca por e-mail, a pequena quantidade retornada e a existência do índice tornaram o acesso direto uma alternativa pertinente. A

distinção entre índice de coluna e índice de expressão, apresentada na documentação do PostgreSQL, ajuda a explicar a mudança observada (PostgreSQL Global Development Group, s.d.a). O resultado não implica que remover funções seja sempre correto: preservar o significado da consulta é condição anterior à comparação de desempenho.

Em conjunto, os casos mostram a contribuição do protótipo como apoio ao diagnóstico. O sistema reúne métricas que selecionam consultas relevantes, planos que explicam operações previstas e sugestões que orientam testes. Essa separação entre diagnóstico e avaliação de ajustes é coerente com a organização discutida por Huang et al. (2023). A contribuição avaliada é a integração dessas etapas em um processo explicável, e não um novo algoritmo de otimização ou uma garantia de ganho para qualquer consulta.

4.4 VERIFICAÇÕES POSTERIORES E INTERFACE

A verificação de 12 de outubro de 2026 confirmou as quantidades das cinco tabelas e o total de 4.648.714 registros. Os cinco indicadores de inconsistência utilizados nessa verificação retornaram zero. Esses resultados complementaram a documentação da base, sem alterar as condições registradas para a primeira rodada.

Na integração posterior da interface, os contadores das consultas experimentais permaneceram com 730 chamadas antes e depois das análises e comparações. Esse resultado foi compatível com a solicitação de planos sem execução das consultas candidatas.

Também foram verificados a rejeição de SQL inválido e o funcionamento da análise de uma consulta sem histórico. Nesse último caso, o sistema apresentou o plano estimado sem atribuir uma duração histórica inexistente.

A interface reuniu as informações em telas de visão geral, análise, comparação e explicação dos conceitos. O usuário pode selecionar uma consulta, observar suas métricas e examinar os sinais encontrados. Na comparação, a forma original e a alternativa são apresentadas lado a lado.

Os sete testes automatizados aprovados e as verificações com o banco indicaram o funcionamento das principais operações. Eles não permitem concluir que a interface foi validada quanto à facilidade de uso, pois não houve estudo com participantes.

4.5 LIMITAÇÕES DOS RESULTADOS

O estudo utilizou dados fictícios, valores de filtros fixos e frequências definidas para a carga. Essas condições permitem observar o comportamento do protótipo, mas não representam todos os cenários encontrados em aplicações reais.

Os tempos podem variar conforme o estado da memória, o armazenamento e os demais processos do computador. A intercalação das versões na busca por e-mail reduziu sua associação com um momento específico da rodada, mas não eliminou esses efeitos.

A primeira rodada não preservou o horário de reinicialização das estatísticas nem uma descrição completa dos processos concorrentes. Seus resultados foram mantidos a partir dos registros de terminal disponíveis. Os arquivos das verificações posteriores pertencem a outro momento e não foram utilizados como se reproduzissem exatamente a rodada original.

A Q12 teve apenas uma execução. A alternativa da consulta por ano foi examinada somente por plano estimado. A busca por e-mail utilizou um valor de entrada e comparou os primeiros conjuntos de resultados, de modo que a equivalência não foi demonstrada para todas as consultas ou entradas possíveis.

O histórico do `pg_stat_statements` é agregado. Consultas com valores diferentes podem compartilhar uma entrada, e a média histórica de linhas pode não representar um caso individual. A diferença entre essa média e a estimativa do plano foi utilizada como sinal de investigação.

Os limites e os pontos das regras foram definidos para o protótipo e não foram calibrados em um conjunto externo de consultas. Por isso, o nível de atenção não possui uma precisão estatística demonstrada. A ferramenta foi desenvolvida para PostgreSQL e uso local; sua aplicação em produção exigiria avaliações adicionais de permissões, carga e validação das entradas.

5 CONCLUSÃO

Este trabalho desenvolveu e avaliou um processo de análise preventiva de consultas SQL baseado em estatísticas históricas e planos de execução. O protótipo reuniu geração de dados, execução de carga, coleta de métricas, identificação de sinais de atenção, comparação de planos e interface web.

Os resultados mostraram que tempo total, tempo médio e

frequência destacam situações diferentes. A consulta de pedidos por ano apresentou o maior tempo acumulado, a leitura integral dos itens apresentou o maior tempo médio registrado, baseado em uma única chamada e a busca por chave primária foi a mais frequente. A quantidade de linhas retornadas também se mostrou necessária para interpretar essas medidas.

Na consulta por ano, a reescrita do filtro aproximou a previsão de linhas da referência histórica. Esse resultado não comprovou redução de tempo, pois a alternativa não foi executada para medição. Na busca por e-mail, a comparação direta permitiu o uso do índice existente e reduziu o tempo médio do servidor de 26,33 para 0,0303 milissegundo nas condições registradas.

Concluiu-se que a combinação de histórico, planos e regras explicáveis pode apoiar a identificação de consultas relevantes e a escolha de alterações a serem testadas. A interface disponibilizou essas informações em uma apresentação visual. O protótipo apoiou a decisão, mantendo a aplicação das mudanças e sua validação sob responsabilidade do usuário.

Como continuidade, recomenda-se repetir os testes com diferentes valores de entrada, volumes e rodadas, ampliar a verificação dos resultados e avaliar outras estratégias de indexação. Também podem ser realizados estudos com usuários para examinar a clareza das explicações e testes em outros SGBDs.

REFERÊNCIAS

ABBASI, M. et al. Optimizing database performance in complex event processing through indexing strategies. **Data**, v. 9, n. 8, p. 93, 2024. Disponível em: <https://doi.org/10.3390/data9080093>. Acesso em: 19 out. 2026.

HUANG, S. et al. Survey on performance optimization for database systems. **Science China Information Sciences**, v. 66, p. 121102, 2023. Disponível em: <https://doi.org/10.1007/s11432-021-3578-6>. Acesso em: 19 out. 2026.

KOSSMANN, J.; PAPENBROCK, T.; NAUMANN, F. Data dependencies for query optimization: A survey. **The VLDB Journal**, v. 31, n. 1, p. 1–22, 2022. Disponível em: <https://doi.org/10.1007/s00778-021-00676-3>. Acesso em: 19 out. 2026.

LAN, H.; BAO, Z.; PENG, Y. A survey on advancing the dbms query optimizer: Cardinality estimation, cost model, and plan enumeration. **Data Science and Engineering**, v. 6, n. 1, p. 86–101, 2021. Disponível em: <https://doi.org/10.1007/s41019-020-00149-7>. Acesso em: 19 out. 2026.

LEIS, V. et al. Still asking: How good are query optimizers, really? **Proceedings of the VLDB Endowment**, v. 18, n. 12, p. 5531–5536, 2025. Disponível em: <https://www.vldb.org/pvldb/vol18/p5531-viktor.pdf>. Acesso em: 19 out. 2026.

PostgreSQL Global Development Group. **PostgreSQL 18 Documentation: Indexes on Expressions**. [S.l.], s.d. Disponível em: <https://www.postgresql.org/docs/18/indexes-expressional.html>. Acesso em: 19 out. 2026.

PostgreSQL Global Development Group. **PostgreSQL 18 Documentation: pg_stat_statements**. [S.l.], s.d. Disponível em: <https://www.postgresql.org/docs/18/pgstatstatements.html>. Acesso em: 19 out. 2026.

PostgreSQL Global Development Group. **PostgreSQL 18 Documentation: Populating a Database**. [S.l.], s.d. Disponível em: <https://www.postgresql.org/docs/18/populate.html>. Acesso em: 19 out. 2026.

PostgreSQL Global Development Group. **PostgreSQL 18 Documentation: Using EXPLAIN**. [S.l.], s.d. Disponível em: <https://www.postgresql.org/docs/18/using-explain.html>. Acesso em: 19 out. 2026.

Python Software Foundation. **Python 3.14 Documentation: time**. [S.l.], s.d. Disponível em: <https://docs.python.org/3.14/library/time.html>. Acesso em: 19 out. 2026.

The Psycopg Team. **Psycopg 3 Documentation: Using COPY to and from tables**. [S.l.], s.d. Disponível em: <https://www.psycopg.org/psycopg3/docs/basic/copy.html>. Acesso em: 19 out. 2026.